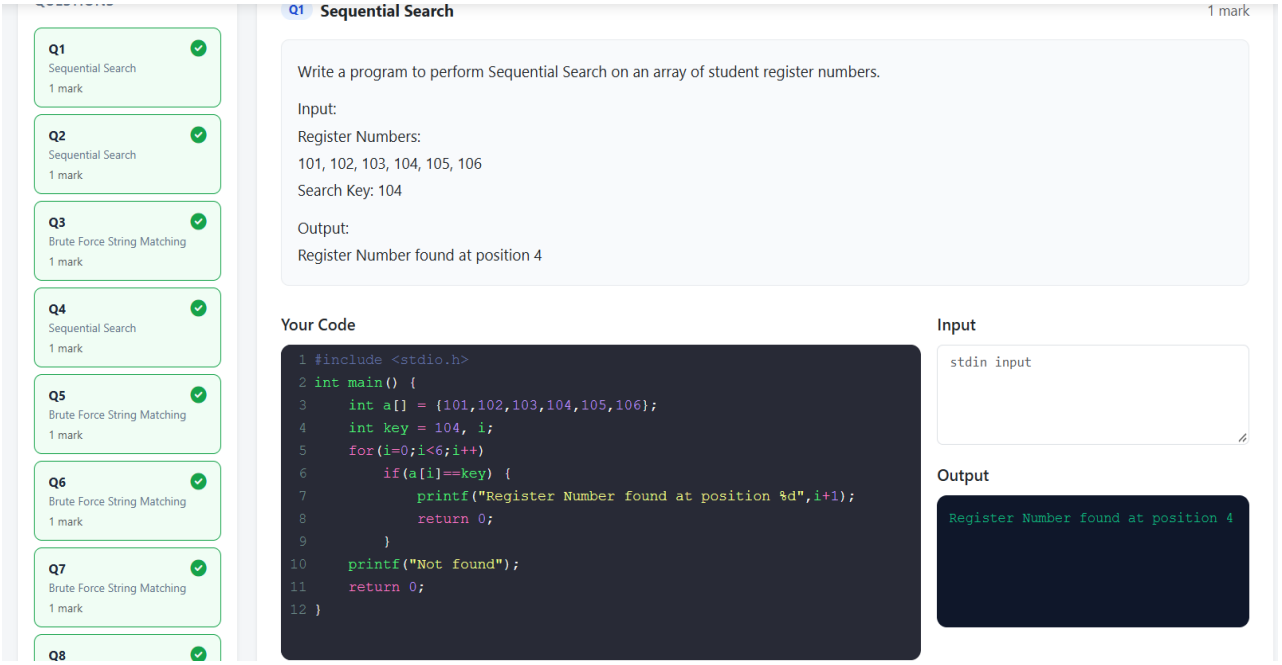
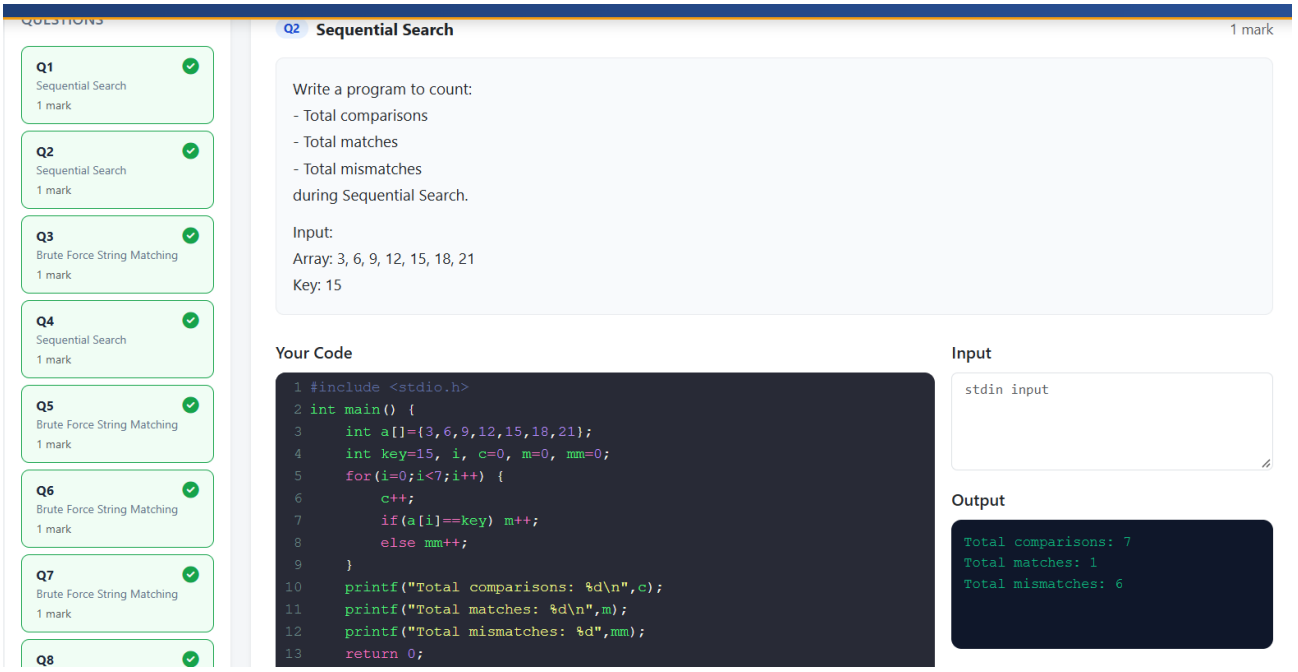
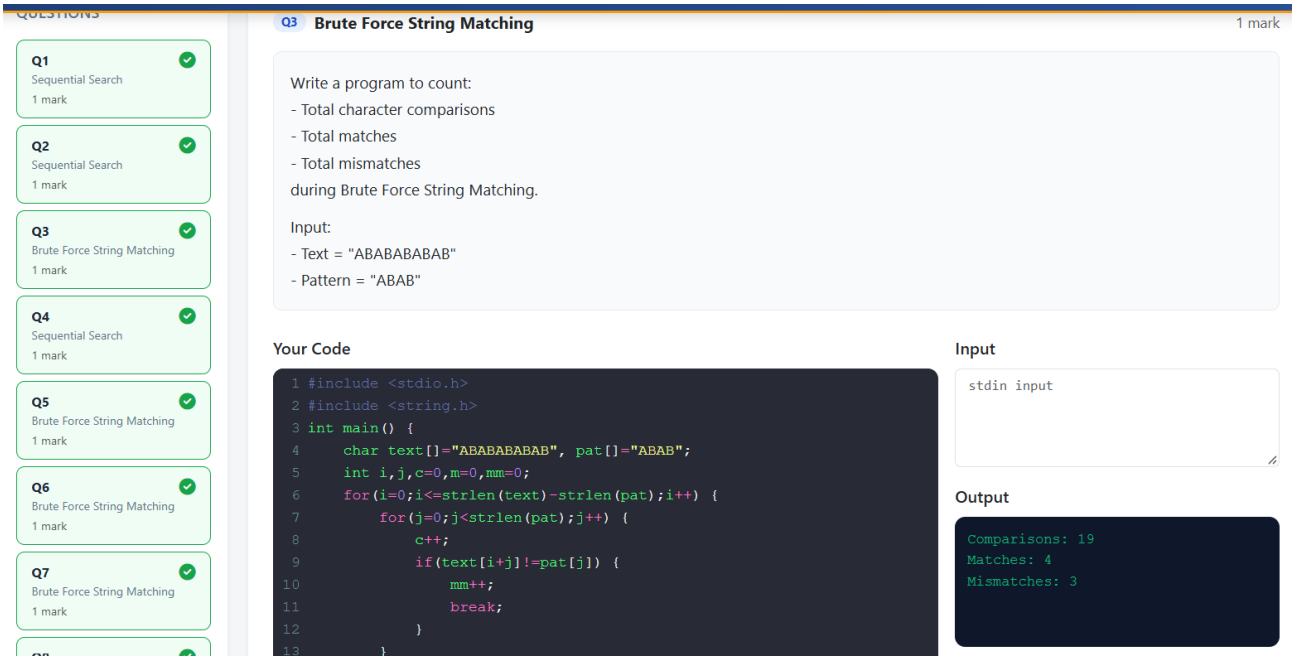
EX:1


Q1 Sequential Search
1 mark
Write a program to perform Sequential Search on an array of student register numbers.
Input:
Register Numbers:
101, 102, 103, 104, 105, 106
Search Key: 104
Output:
Register Number found at position 4
Your Code
#include <stdio.h>
int main() {
int a[] = {101,102,103,104,105,106};
int key = 104, i;
for(i=0;i<6;i++)
if(a[i]==key) {
printf("Register Number found at position %d",i+1);
return 0;
}
printf("Not found");
return 0;
}
Input
stdin input
Output
Register Number found at position 4

EX:2


Q2 Sequential Search
1 mark
Write a program to count:
- Total comparisons
- Total matches
- Total mismatches
during Sequential Search.
Input:
Array: 3, 6, 9, 12, 15, 18, 21
Key: 15
Your Code
#include <stdio.h>
int main() {
int a[]={3,6,9,12,15,18,21};
int key=15, i, c=0, m=0, mm=0;
for(i=0;i<7;i++) {
c++;
if(a[i]==key) m++;
else mm++;
}
printf("Total comparisons: %d\n",c);
printf("Total matches: %d\n",m);
printf("Total mismatches: %d",mm);
return 0;
Input
stdin input
Output
Total comparisons: 7
Total matches: 1
Total mismatches: 6

EX:3



Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Brute Force String Matching 1 mark
Q7 Brute Force String Matching 1 mark

Q3 **Brute Force String Matching** 1 mark

Write a program to count:
- Total character comparisons
- Total matches
- Total mismatches

during Brute Force String Matching.

Input:
- Text = "ABABABABAB"
- Pattern = "ABAB"

Your Code

```c
#include <stdio.h>
#include <string.h>
int main() {
    char text[]="ABABABABAB", pat[]="ABAB";
    int i,j,c=0,m=0,mm=0;
    for(i=0;i<=strlen(text)-strlen(pat);i++) {
        for(j=0;j<strlen(pat);j++) {
            c++;
            if(text[i+j]!=pat[j]) {
                mm++;
                break;
            }
        }
```

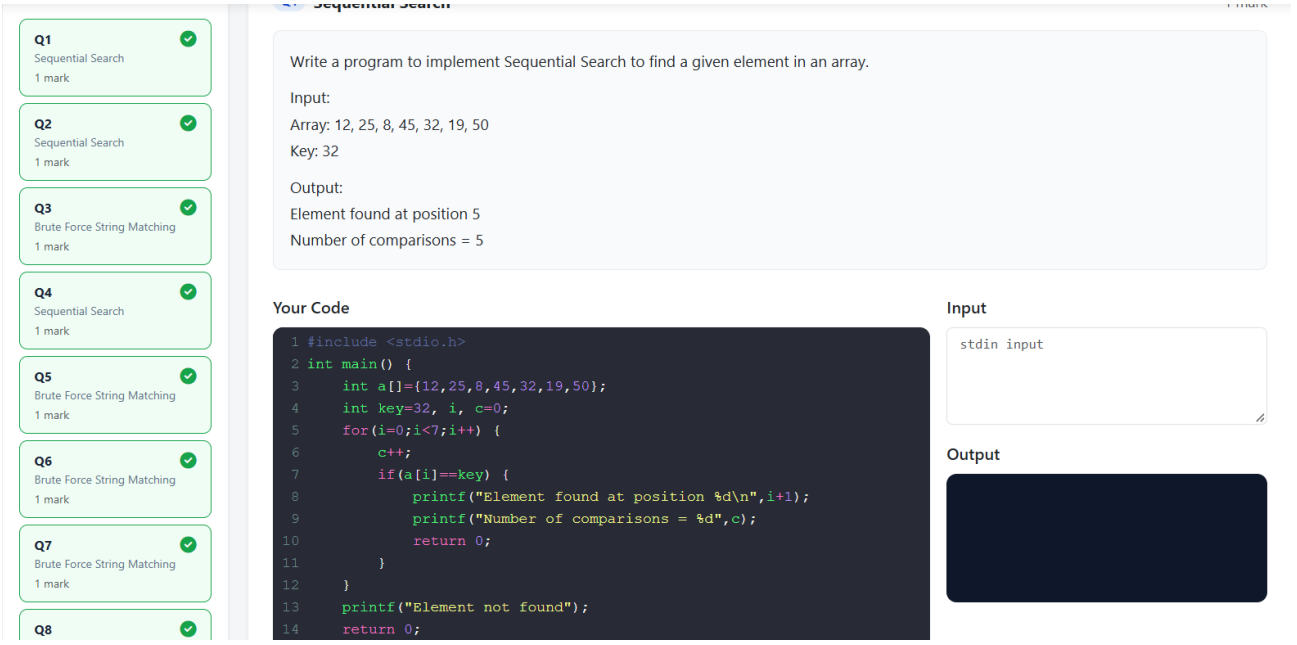
Input

stdin input

Output

```
Comparisons: 19
Matches: 4
Mismatches: 3
```

EX:4



Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Brute Force String Matching 1 mark
Q7 Brute Force String Matching 1 mark
Q8

Write a program to implement Sequential Search to find a given element in an array.

Input:
Array: 12, 25, 8, 45, 32, 19, 50
Key: 32

Output:
Element found at position 5
Number of comparisons = 5

Your Code

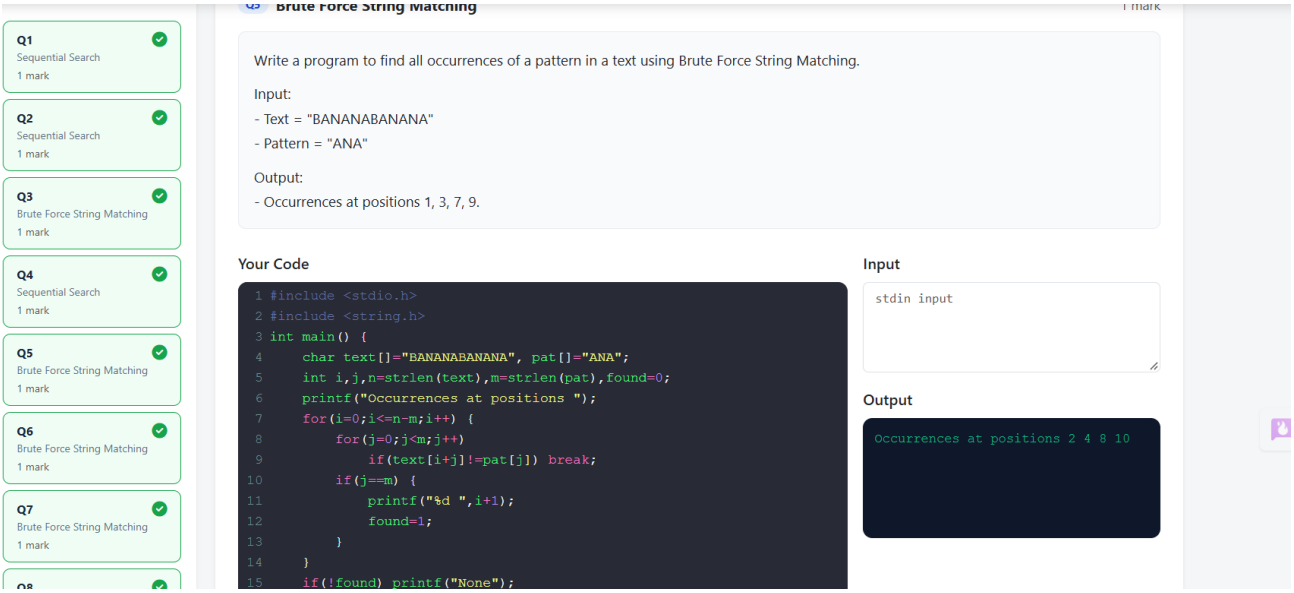
```c
#include <stdio.h>
int main() {
    int a[]={12,25,8,45,32,19,50};
    int key=32, i, c=0;
    for(i=0;i<7;i++) {
        c++;
        if(a[i]==key) {
            printf("Element found at position %d\n",i+1);
            printf("Number of comparisons = %d",c);
            return 0;
        }
    }
    printf("Element not found");
    return 0;
```

Input

stdin input

Output

EX:5



Q1 Sequential Search 1 mark
Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Brute Force String Matching 1 mark
Q7 Brute Force String Matching 1 mark

Brute Force String Matching — 1 mark

Write a program to find all occurrences of a pattern in a text using Brute Force String Matching.

Input:
- Text = "BANANABANANA"
- Pattern = "ANA"

Output:
- Occurrences at positions 1, 3, 7, 9.

Your Code

```c
#include <stdio.h>
#include <string.h>
int main() {
    char text[]="BANANABANANA", pat[]="ANA";
    int i,j,n=strlen(text),m=strlen(pat),found=0;
    printf("Occurrences at positions ");
    for(i=0;i<=n-m;i++) {
        for(j=0;j<m;j++)
            if(text[i+j]!=pat[j]) break;
        if(j==m) {
            printf("%d ",i+1);
            found=1;
        }
    }
    if(!found) printf("None");
```

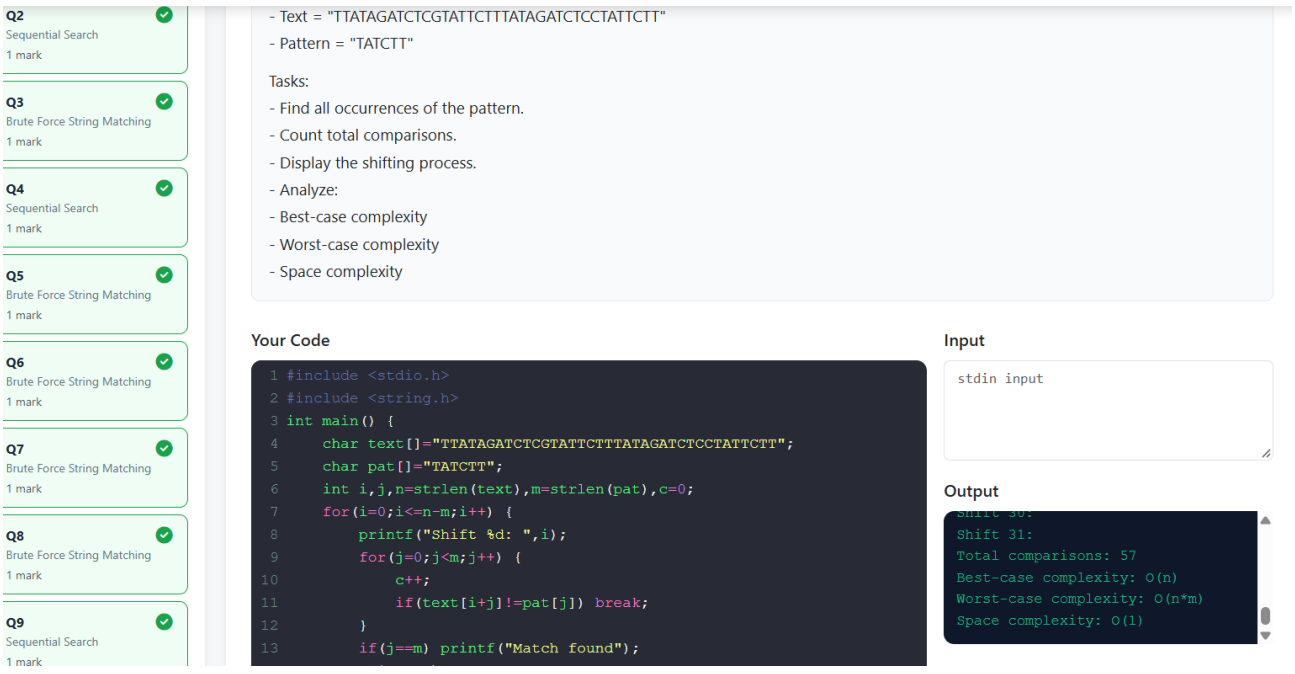
Input

stdin input

Output

```
Occurrences at positions 2 4 8 10
```

EX:6



Q2 Sequential Search 1 mark
Q3 Brute Force String Matching 1 mark
Q4 Sequential Search 1 mark
Q5 Brute Force String Matching 1 mark
Q6 Brute Force String Matching 1 mark
Q7 Brute Force String Matching 1 mark
Q8 Brute Force String Matching 1 mark
Q9 Sequential Search 1 mark

- Text = "TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT"
- Pattern = "TATCTT"

Tasks:
- Find all occurrences of the pattern.
- Count total comparisons.
- Display the shifting process.
- Analyze:
- Best-case complexity
- Worst-case complexity
- Space complexity

Your Code

```c
#include <stdio.h>
#include <string.h>
int main() {
    char text[]="TTATAGATCTCGTATTCTTTATAGATCTCCTATTCTT";
    char pat[]="TATCTT";
    int i,j,n=strlen(text),m=strlen(pat),c=0;
    for(i=0;i<=n-m;i++) {
        printf("Shift %d: ",i);
        for(j=0;j<m;j++) {
            c++;
            if(text[i+j]!=pat[j]) break;
        }
        if(j==m) printf("Match found");
```

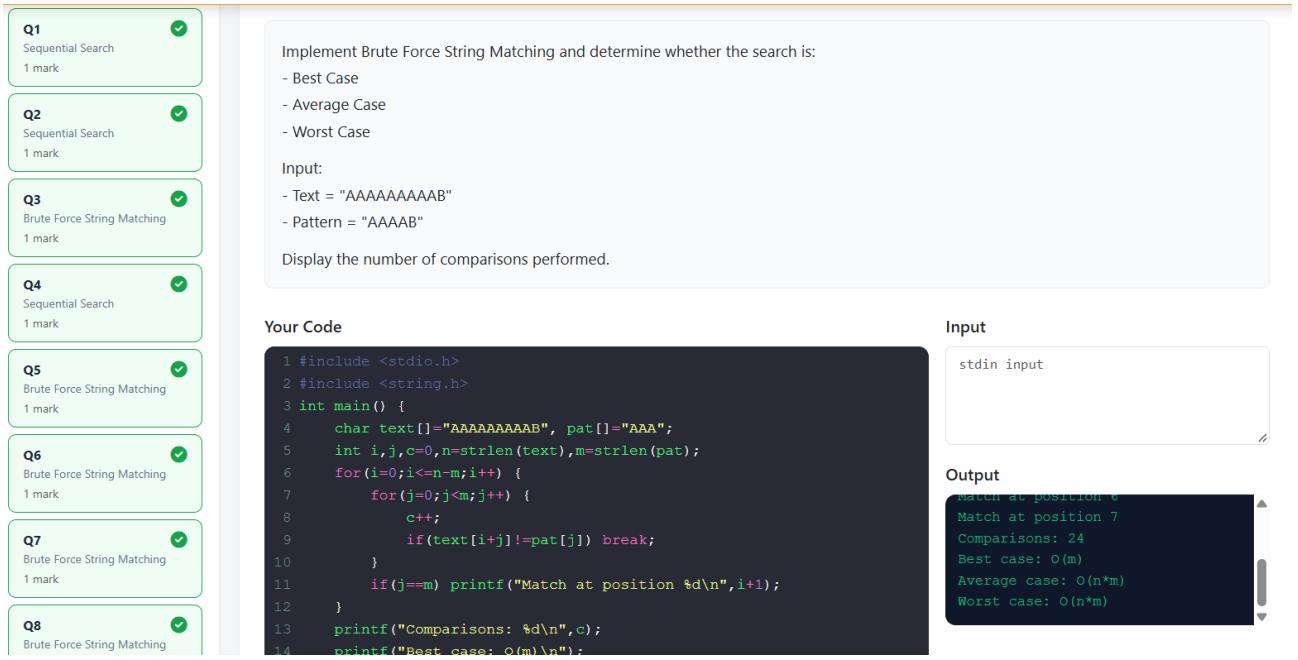
Input

stdin input

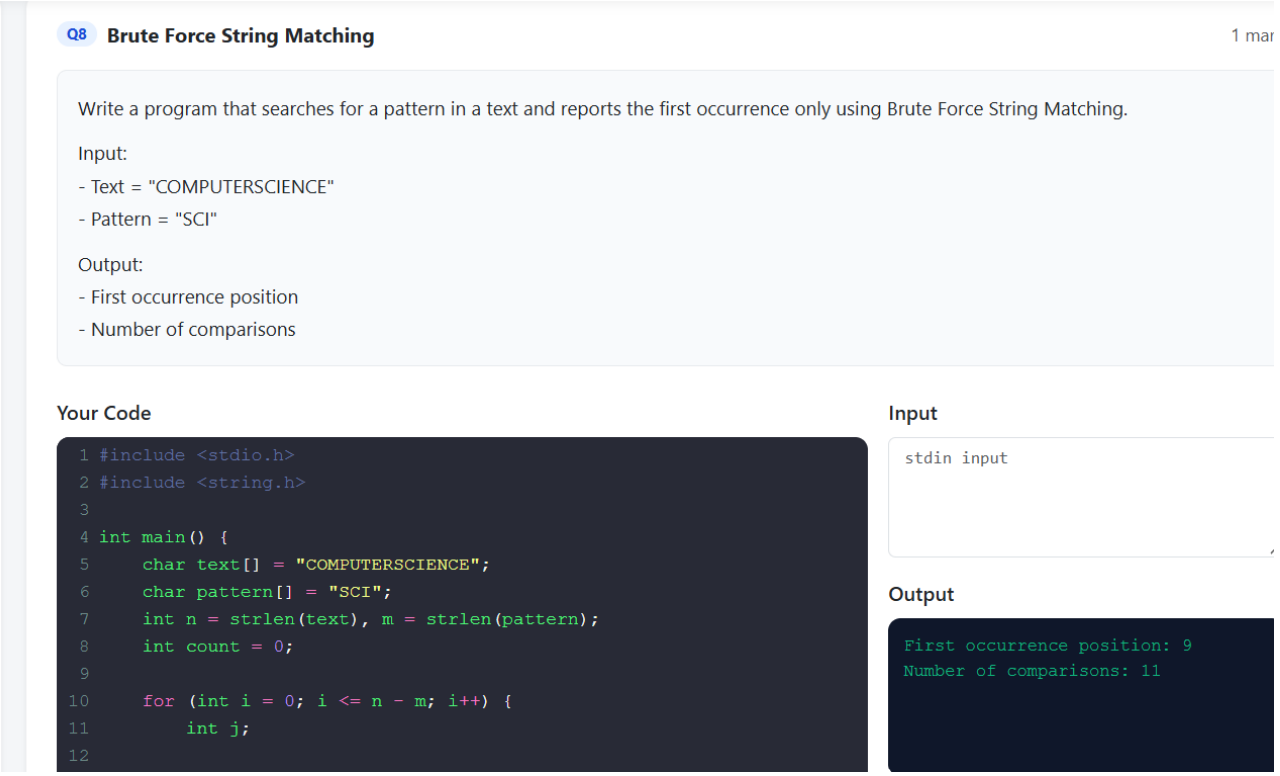
Output

```
Shift 31:
Total comparisons: 57
Best-case complexity: O(n)
Worst-case complexity: O(n*m)
Space complexity: O(1)
```

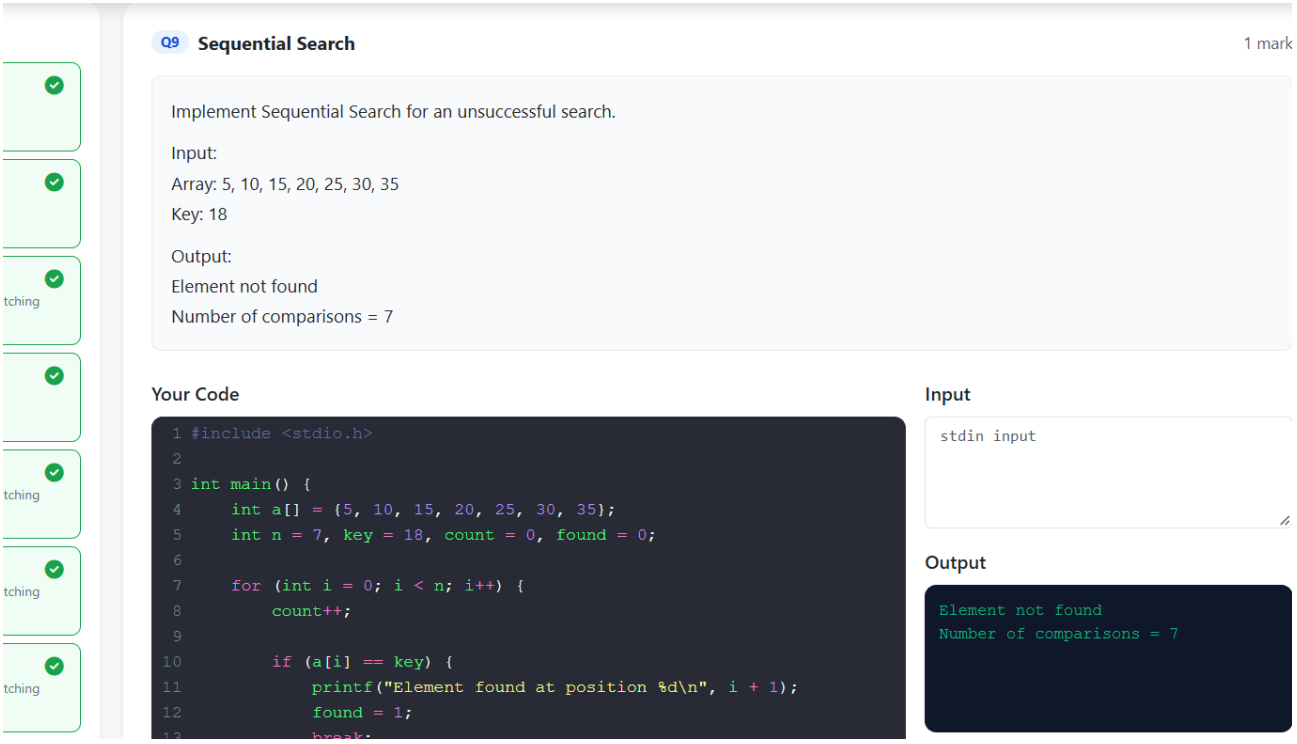
EX:7


Q1
Sequential Search
1 mark
Q2
Sequential Search
1 mark
Q3
Brute Force String Matching
1 mark
Q4
Sequential Search
1 mark
Q5
Brute Force String Matching
1 mark
Q6
Brute Force String Matching
1 mark
Q7
Brute Force String Matching
1 mark
Q8
Brute Force String Matching
Implement Brute Force String Matching and determine whether the search is:
- Best Case
- Average Case
- Worst Case
Input:
- Text = "AAAAAAAAAB"
- Pattern = "AAAAB"
Display the number of comparisons performed.
Your Code
#include <stdio.h>
#include <string.h>
int main() {
char text[]="AAAAAAAAAB", pat[]="AAA";
int i,j,c=0,n=strlen(text),m=strlen(pat);
for(i=0;i<=n-m;i++) {
for(j=0;j<m;j++) {
c++;
if(text[i+j]!=pat[j]) break;
}
if(j==m) printf("Match at position %d\n",i+1);
}
printf("Comparisons: %d\n",c);
Input
stdin input
Output
Match at position 7
Comparisons: 24
Best case: O(m)
Average case: O(n*m)
Worst case: O(n*m)

EX:8


Q8 Brute Force String Matching
1 mar
Write a program that searches for a pattern in a text and reports the first occurrence only using Brute Force String Matching.
Input:
- Text = "COMPUTERSCIENCE"
- Pattern = "SCI"
Output:
- First occurrence position
- Number of comparisons
Your Code
#include <stdio.h>
#include <string.h>
int main() {
char text[] = "COMPUTERSCIENCE";
char pattern[] = "SCI";
int n = strlen(text), m = strlen(pattern);
int count = 0;
for (int i = 0; i <= n - m; i++) {
int j;
Input
stdin input
Output
First occurrence position: 9
Number of comparisons: 11

EX:9



Q9 **Sequential Search** 1 mark

Implement Sequential Search for an unsuccessful search.

Input:
Array: 5, 10, 15, 20, 25, 30, 35
Key: 18

Output:
Element not found
Number of comparisons = 7

Your Code

```c
#include <stdio.h>

int main() {
    int a[] = {5, 10, 15, 20, 25, 30, 35};
    int n = 7, key = 18, count = 0, found = 0;

    for (int i = 0; i < n; i++) {
        count++;

        if (a[i] == key) {
            printf("Element found at position %d\n", i + 1);
            found = 1;
            break;
```

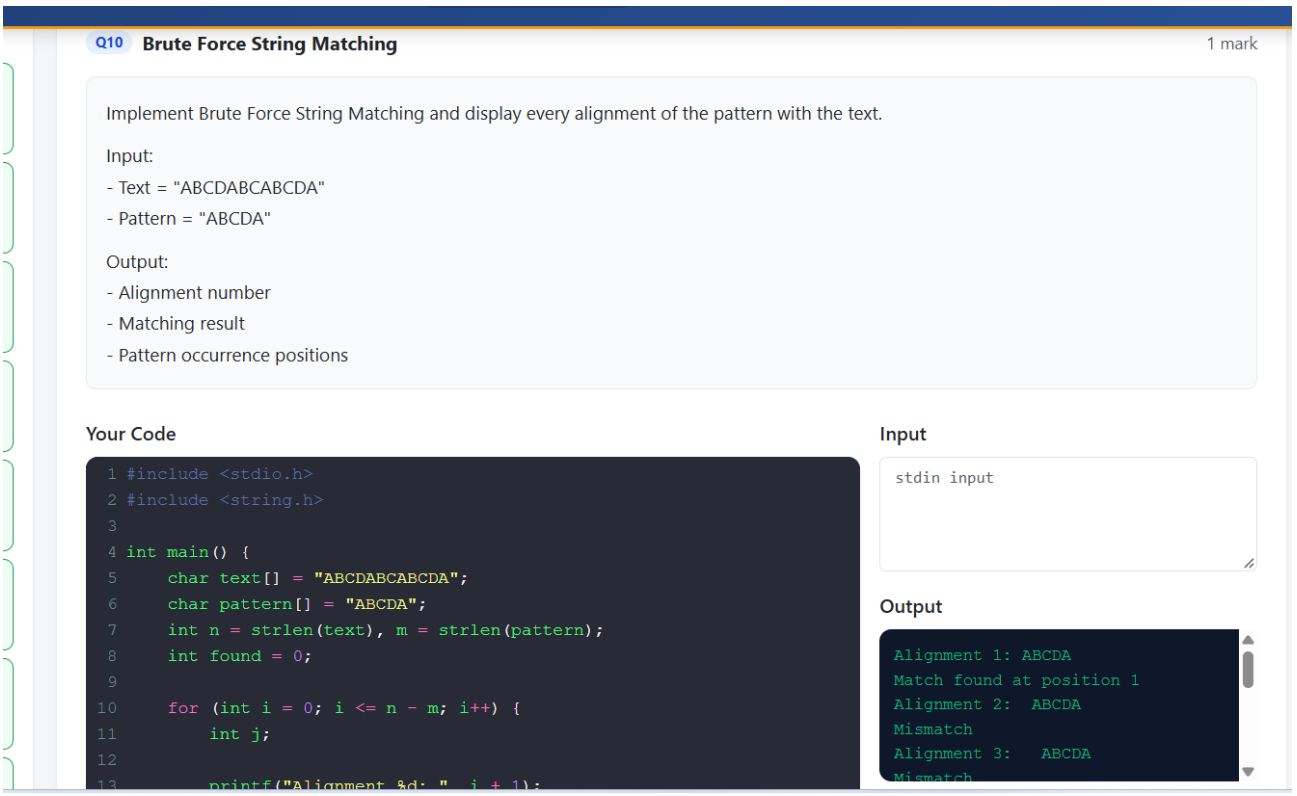
Input

```
stdin input
```

Output

```
Element not found
Number of comparisons = 7
```

EX:10



Q10 **Brute Force String Matching** 1 mark

Implement Brute Force String Matching and display every alignment of the pattern with the text.

Input:
- Text = "ABCDABCABCDA"
- Pattern = "ABCDA"

Output:
- Alignment number
- Matching result
- Pattern occurrence positions

Your Code

```c
#include <stdio.h>
#include <string.h>

int main() {
    char text[] = "ABCDABCABCDA";
    char pattern[] = "ABCDA";
    int n = strlen(text), m = strlen(pattern);
    int found = 0;

    for (int i = 0; i <= n - m; i++) {
        int j;

        printf("Alignment %d: ", i + 1);
```

Input

```
stdin input
```

Output

```
Alignment 1: ABCDA
Match found at position 1
Alignment 2:  ABCDA
Mismatch
Alignment 3:   ABCDA
Mismatch
```

EX:11

73
18
100

Tasks:
- Show each comparison.
- Count comparisons for every search.
- Identify best, average, and worst cases.
- Determine:
- Best-case complexity
- Average-case complexity
- Worst-case complexity
- Space complexity

Your Code

```c
#include <stdio.h>

int main() {
    int a[] = {45, 23, 67, 12, 89, 34, 56, 78,
               90, 11, 29, 73, 18, 64, 37};
    int n = 15, keys[] = {73, 18, 100};

    for (int k = 0; k < 3; k++) {
        int count = 0, pos = -1;

        printf("\nSearch key: %d\n", keys[k]);
```

Input

stdin input

Output

```
Best case: O(1)
Average case: O(n)
Worst case: O(n)
Space complexity: O(1)
```

EX:12

Q12 **Brute Force String Matching** 1 mark

Write a program to compare the execution of Brute Force String Matching for:
- Successful search
- Unsuccessful search

Input:
- Text = "PROGRAMMINGLAB"
- Pattern 1 = "LAB"
- Pattern 2 = "TEST"

Display the comparison count for both cases.

Your Code

```python
def brute_force_string_match(text, pattern):
    n = len(text)
    m = len(pattern)
    comparisons = 0
    position = -1

    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
```

Input

stdin input

Output

```
Unsuccessful Search:
Pattern = 'TEST'
Pattern not found
Number of comparisons = 11
```

EX:13

Implement the Brute Force String Matching Algorithm to search for a pattern in a given text.

Input:
- Text = "AABAACAADAABAABA"
- Pattern = "AABA"

Output:
- Position(s) where the pattern occurs.
- Total number of comparisons.

Your Code

```python
def brute_force_string_match(text, pattern):
    n = len(text)
    m = len(pattern)
    positions = []
    comparisons = 0

    for i in range(n - m + 1):
        j = 0
        while j < m:
            comparisons += 1
            if text[i + j] != pattern[j]:
                break
```

Input

```
stdin input
```

Output

```
Pattern found at position(s): [0, 9, 12]
Total number of comparisons: 30
```

EX:14

Q14 **Sequential Search** 1 mark

Write a program to find the first occurrence of an element using Sequential Search.

Input:
Array: 10, 25, 15, 25, 30, 25, 40
Key: 25

Output:
First occurrence at position 2

Your Code

```c
#include <stdio.h>
int main(){
  int a[] = {10, 25, 15, 25, 30, 25, 40};
    int n = 7, key = 25;
    for (int i = 0; i<n; i++){
        if (a[i] == key){
            printf("First occurrence at position %d\n",i + 1);
            return 0;
        }
        }
         printf("Element not found\n");
        return 0;
    }
```

Input

```
stdin input
```

Output

```
First occurrence at position 2
```

EX:15

Q15 Sequential Search 1

Implement Sentinel Sequential Search and compare it with ordinary Sequential Search.

Input:
Array: 14, 9, 22, 35, 18, 41, 27
Key: 18

Output:
Position found
Comparison count

Your Code

```
a = [14, 9, 22, 35, 18, 41, 27]
key = 18

for i in range(len(a)):
    if a[i] == key:
        print("Found at position",i +1)
        break
    else:
        print("Not found")
```

Input

```
stdin input
```

Output

```
Not found
Not found
Not found
Not found
Found at position 5
```

EX:16

Q16 Sequential Search 1 mark

Write a program to perform Sequential Search on a 2-D matrix.

Input:
12 8 15
5 18 27
9 11 24

Key = 24

Output:
Element found at Row 3 Column 3

Your Code

```
a = [[12 ,8, 15],[5, 18, 27],[9, 11, 24]]
key = 24
for i in range(3):
    for j in range(3):
        if a[i][j] == key:
            print("Element found at ",i +1,j + 1)
```

Input

```
stdin input
```

Output

```
Element found at  3 3
```

EX:17

Q17 **Brute Force String Matching** 1 mark

Implement Brute Force String Matching for case-insensitive searching.

Input:
- Text = "DataStructuresAndAlgorithms"
- Pattern = "ALGORITHMS"

Output:
- Pattern found position

Your Code

```
1 text = "DataStructures and Algorithms"
2 pattern = "Algorithma"
3 if pattern.lower()in text.lower():
4     print("Pattern found")
5 else:
6     print("Pattern not found")
```

Input

```
stdin input
```

Output

```
Pattern not found
```

EX:18

Q18 **Sequential Search** 1 mark

Implement Sequential Search for string data.

Input:
Names:
Anu, Bala, Charan, Deepa, Esha, Farhan

Search Name:
Deepa

Output:
Name found at position 4

Your Code

```
1 def sequential_search(names, target):
2     for index, name in enumerate(names):
3         if name == target:
4             return index + 1  # position starts from 1
5     return -1
6
7 # Input
8 names_input = input().strip()
9 search_name = input().strip()
10
11 names = [name.strip() for name in names_input.split(',')]
12
```

Input

```
Anu, Bala, Charan, Deepa, Esha, Farhan
Deepa
```

Output

```
Name found at position 4
```

EX:19

Q19 Sequential Search — 1 mark

Implement Sequential Search to find all occurrences of a given element.

Input:
Array: 7, 12, 7, 25, 18, 7, 30, 7
Key: 7

Output:
Occurrences at positions:
1, 3, 6, 8
Total occurrences = 4

Your Code

```c
#include <stdio.h>
int main(){
    int a[] = {7, 12, 7, 25,18, 7, 30, 7};
    int n = 8, key = 7, count = 0;
    printf("Occurrences at position:");
    for(int i = 0; i<n; i++){
        if (a[i] == key){
            printf("%d",i + 1);
            count++;
        }
    }
    printf("\nTotal occurrences = %d\n",count);
```

Input

```
stdin input
```

Output

```
Occurrences at position:1368
Total occurrences = 4
```

EX:20

Q20 Brute Force String Matching — 1 mark

Implement Brute Force String Matching and display the comparison table for each alignment.

Input:
- Text = "MISSISSIPPI"
- Pattern = "ISSI"

Output:
- Shift number
- Comparisons made
- Match/Mismatch result

Your Code

```c
#include <stdio.h>
int main() {
    int a[] = {101,102,103,104,105,106};
    int key = 104, i;
    for(i=0;i<6;i++)
        if(a[i]==key) {
            printf("Register Number found at position %d",i+1);
            return 0;
        }
    printf("Not found");
    return 0;
}
```

Input

```
stdin input
```

Output

```
Register Number found at position 4
```